WordPress-01 第18回_まとめ

第18回の目標

- **WordPressとは何かを知る**
- **WebサイトをWordPress化する準備を知る**
- **Webサイトのテンプレート化の流れをつかむ**

WordPressとは

- ブログからWebサイトまで作れる、オープンソースのソフトウェア
- **CMS**（コンテンツ管理システム）の1つ。HTML/CSSで作ったサイトを、管理画面から操作できるようにすることを「WordPress化（CMS化）」という
- 一番の特徴は「**管理画面からWebサイトを編集できる**」こと
  - ページのタイトルや文章の編集、お知らせの更新、下層ページの追加など
  - 普通のHTMLサイトは文言を変えるだけでもファイルの編集が必要
- ZeroPlus MediaもWordPressで作られている

WordPressの仕組み

技術	役割
MySQL	情報を整理して保存する「データベース」を作るシステム
PHP	データベースへの保存・取得が得意なプログラミング言語

- 管理画面で入力した情報が、PHPを通じてデータベースに保存され、サイトがそれを読み込んで表示が変わる
- 最初は、PHPの基本構文と、プログラミングの基礎（関数・引数・配列など）が分かれば十分

WordPressを使う・使わない場面

使うとよい場面	使わなくてもよい場面
更新が多いサイト	更新が少なく、変更は制作会社に頼む運用
クライアントが文言をよく変えたいサイト	コンテンツを増やす予定のない、名刺代わりのシンプルなサイト

使うとよい場面	使わなくてもよい場面
投稿・固定ページなどの機能を使いたいサイト	表示速度や高いセキュリティを最優先するサイト

- 注意点：配布テーマをそのまま使うと細かいデザインの要望に応えにくい、機能が多いと重くなる、セキュリティに注意が必要
- WordPressで何ができるか学ぼう

WordPress化の準備

① 環境構築（Local）

WordPressを自分のPCで動かす環境を作る。MAMP・XAMPPなどの方法もあるが、授業では**Local**を使う。

1. Localをダウンロードしてインストールする
    - Macは「このMacについて」でチップを確認し、M1〜M4なら「Mac Apple Silicon」、Intelなら「Mac Intel」を選ぶ（Intelは古く、Localが重くなりやすい）
2. 「CREATE A NEW SITE」でサイトを作る
    - 名前：ZeroPlus／バージョン：Preferred／ユーザー名とパスワード：練習用なので覚えやすいもの
    - パソコン自体のパスワードを聞かれたら、WordPressではなくPCのパスワードを入れる
3. Localの「WP Admin」から管理画面にログインする（admin＝管理者）
4. 「Settings」で言語を日本語、タイムゾーンをUTC+9にする

- Localを立ち上げるとPCが重くなることがあるので、授業前に再起動しておくとよい

② テーマフォルダをthemesに入れる

「Site folder」から`app > public > wp-content > themes`の順に開き、WordPress化するフォルダを入れる。

構造	判定
themes ＞ フォルダ ＞ 中身（front-page.php など）	○ 一番シンプル
themes ＞ フォルダ ＞ フォルダ ＞ 中身	○ 教材はこの形
themes ＞ フォルダ ＞ フォルダ ＞ フォルダ ＞ 中身	✕ 階層が深すぎてテーマとして認識されないことが多い

- **themesフォルダに入れないと、編集しても反映されない**
- 授業では、themesに入れたフォルダをVSCodeにドラッグ＆ドロップして開く

③ テーマを有効化する

管理画面の「外観」→「テーマ」で、入れたテーマを有効化する。

テーマ化できないときの確認 - テーマフォルダの直下（front-page.phpと同じ場所）に`style.css`と`index.php`があるか - `style.css`：テーマの情報を書くファイル。cssフォルダのスタイル用のファイルとは別に用意する - `index.php`：中身は空でよい - この2つがないと「壊れているテーマ」と表示される - 階層が深すぎないか

```
/* style.css の冒頭に書くテーマ情報（管理画面の「外観＞テーマ」に表示される） */
@charset "UTF-8";
/*
Theme Name: テーマ名
Author: テーマ作者名
Description: テーマの概要
Version: テーマのバージョン
*/
```

- WordPressを勉強する準備をしよう（ツールバーの非表示、インデックスの設定もしておく）

テンプレートファイルとテンプレート階層

よく作るファイル

ファイル	役割
`index.php`	テーマに必須。他に当てはまるファイルがないときに最終的に使われる
`style.css`	テーマに必須。テーマ名などを書く
`functions.php`	テーマ・プラグイン・WordPress関数・管理画面などをカスタマイズする。間違えると画面が表示されなくなることがある。`function.php`と書き間違えやすい
`404.php`	ページが見つからないときに表示する
`screenshot.png`	「外観＞テーマ」の一覧に出るテーマの画像
`front-page.php`	トップページ（授業ではコーポレートサイトのトップ）
`home.php`	投稿一覧ページ（授業ではブログのトップ）
`header.php` / `footer.php` / `sidebar.php`	ヘッダー・フッター・サイドバー

テンプレート階層

- WordPressは、アクセスされたURLから自動でテンプレートファイルを選んで表示する
- 候補が複数あるとき、優先順位の高いファイルを選ぶ仕組みが**テンプレート階層**
- ファイル名はこの階層図に基づいて決める（📎 テンプレート階層（WordPress公式）／テンプレート階層と優先順位の早見表）
- 📎 front-page.phpとhome.phpとindex.phpの違い

PHPの基本

WordPressはPHPでできているので、テーマもPHPで作る。PHPはHTMLの中に埋め込んで書く。

書き方	意味
`<?php 処理 ?>`	PHPの処理を囲む
`echo "文字";`	文字を表示する
`'Zero' . 'Plus'`	文字の連結（`+`ではなく`.`）
`$name = "ZeroPlus";`	変数（頭に`$`）
`["りんご", "ばなな"]`	配列（`array()`でもよい。プロジェクト内で統一する）
`["apple" => "りんご"]`	連想配列（キーと値の組。WordPressの設定でよく使う）
`// 1行` `/* 複数行 */`	コメント

```
<?php if (is_front_page()) : ?>
  <!-- トップページのときの処理 -->
<?php else : ?>
  <!-- それ以外のときの処理 -->
<?php endif; ?>

<?php while (条件) : ?>
  <!-- 繰り返す処理（メインループで使う） -->
<?php endwhile; ?>

<?php foreach ($配列 as $値) : ?>
  <!-- 配列の要素を1つずつ処理する -->
<?php endforeach; ?>
```

- WordPressでは、`{}`ではなく`:`と`endif;`などを使う書き方がよく使われる

- `<?php ?>`を毎回書くのは大変なので、スニペット登録しておく（html.json）

```
{
  "php": {
    "prefix": "php",
    "body": ["<?php $1 ?>"],
    "description": "php tag"
  }
}
```

- WordPressを学ぶにあたってPHPの基本を学ぼう

テンプレート化の流れ

1. 静的なHTML（`index.html`）を丸ごとコピーして、`front-page.php`に貼る
2. ヘッダー・フッターなどの共通部分を切り取って、テンプレートパーツファイルに分ける
3. 切り分けた部分を、WordPress関数で呼び出す
4. 相対パスを、WordPress関数を使った絶対パスに書き換える（エスケープも忘れずに）
5. CSS・Webフォントを`functions.php`で読み込む

テンプレートパーツファイル

複数ページに共通する部分を1つのファイルに切り出して、使いたい場所で呼び出す仕組み。

ファイル	切り出す範囲	呼び出し方
`header.php`	`<!DOCTYPE html>`から`</header>`まで	`<?php get_header(); ?>`
`footer.php`	`<footer>`から`</html>`まで	`<?php get_footer(); ?>`
`sidebar.php`	サイドバー（`aside`）	`<?php get_sidebar(); ?>`
`header-blog.php`	2種類目のヘッダー（名前付き）	`<?php get_header('blog'); ?>`
独自のパーツ（`cta.php`など）	CTAなどの繰り返し部分	`<?php get_template_part('cta'); ?>`

- 独自パーツは`template-parts`フォルダにまとめることが多い（`get_template_part('template-parts/content')`）
- ヘッダーの`</head>`の直前に`<?php wp_head(); ?>`、フッターの`</body>`の直前に`<?php wp_footer(); ?>`を書く。WordPressやプラグインが必要なHTMLを出力するので、ないとプラグインが動かないことがある
- WordPressのテンプレートパーツファイルについて押さえよう

URL出力関数で絶対パスにする

関数	取得するもの	使う場所
`get_template_directory_uri()`	有効なテーマのフォルダのURL（末尾に`/`はつかない）	画像・favicon・CSSのパス
`home_url()`	サイトのトップのURL。引数で下層ページも指定できる	サイト内のリンク

```
<!-- 画像：テーマフォルダの中の画像を読み込む -->
<img src="<?php echo esc_url(get_template_directory_uri() . '/img/sample.jpg'); ?>" alt="" />

<!-- リンク：トップページと下層ページ -->
<a href="<?php echo esc_url(home_url()); ?>">トップ</a>
<a href="<?php echo esc_url(home_url('/blog')); ?>">ブログ</a>
```

- WordPressでよく使うURL出力関数を学ぼう

エスケープ

- 入力内容に含まれるコードがそのまま実行されないよう、文字を別の表現に変える処理（サニタイジングともいう）。例：`<` → `<`
- **XSS**（クロスサイトスクリプティング）対策。仕込まれたスクリプトが実行されると、入力情報やCookieが漏れるおそれがある

関数	使う場所
`esc_url()`	`href`や`src`に入れるURL（`get_template_directory_uri()`・`home_url()`と一緒に使う）
`esc_html()`	文字の出力（カスタムフィールドの値など）
`esc_attr()`	`class`や`alt`などの属性の値

- `the_`で始まる関数は中でエスケープされるので不要。`get_`で始まる関数は値を取るだけなので、`echo`するときにエスケープする
- WordPressの特定の関数をエスケープしよう

functions.phpでCSSを読み込む

WordPressでは、CSSやWebフォントは`head`に`link`で書かず、`functions.php`にまとめて読み込む。

```php
<?php
// 読み込むタイミングを指定する（wp_enqueue_scripts＝管理画面以外で実行）
add_action('wp_enqueue_scripts', 'add_styles');

function add_styles() {
  // Google Fonts（外部CSSはURLをそのまま指定）
  wp_enqueue_style('google-fonts_style', 'https://fonts.googleapis.com/css2?family=Noto+Sans+JP:wght@400;700&display=swap');

  // reset.cssは登録だけしておく（まだ読み込まない）
  wp_register_style(
    'reset_style',                                    // 識別名
    get_template_directory_uri() . '/css/reset.css',  // CSSの絶対パス
    array(),                                          // 先に読み込むCSS
    '1.0'                                             // バージョン
  );

  if (is_front_page()) {
    // トップページ：reset.css → corporate.css の順に読み込む
    wp_enqueue_style('corporate_style', get_template_directory_uri() . '/css/corporate.css', array('reset_style'), '1.0');
  } else {
    // それ以外：reset.css → blog.css の順に読み込む
    wp_enqueue_style('blog_style', get_template_directory_uri() . '/css/blog.css', array('reset_style'), '1.0');
  }
}
```

関数	動き
`add_action(フック, 関数名)`	関数を実行するタイミングを指定する。`functions.php`の記述は管理画面にも影響するので、タイミングを指定する
`wp_register_style()`	登録だけして、読み込まない
`wp_enqueue_style()`	登録して読み込む

引数	意味
1つ目（識別名）	スタイルの名前
2つ目（パス）	CSSの絶対パス

引数	意味
3つ目（依存先）	先に読み込む必要があるCSSの識別名の配列。ここに書いた登録済みのCSSも一緒に読み込まれる
4つ目（バージョン）	変更したときに数字を変えると、キャッシュで反映されない問題を防げる

- `register`で登録しただけのCSSは、他のCSSの依存先に指定しないと読み込まれない。条件によって切り替えるときは「register → enqueue」と分けると便利
- WordPressでCSSを読み込もう

条件分岐でページを判別する

関数	trueになるページ
`is_front_page()`	サイトのトップページ
`is_home()`	投稿一覧ページ（ブログのトップ）
`is_page()`	固定ページ（引数でスラッグやIDを指定できる。例：`is_page('company')`）
`is_archive()`	アーカイブページ（カテゴリー・タグ・日付・カスタム投稿など）
`is_category()`	カテゴリーページ
`is_404()`	404ページ

- 表示設定で「固定ページ」を選び、ホームページと投稿ページを別に指定すると、トップで`is_front_page()`、投稿一覧で`is_home()`がtrueになる
- WordPressの条件分岐でページを判別しよう

管理画面の設定

表示設定（トップとブログを分ける）

1. 固定ページで「TOP」を作って公開する
2. 「ブログ」を作り、パーマリンク（URL）を`blog`にして公開する
3. 「設定」→「表示設定」→「ホームページの表示」で「固定ページ」を選ぶ
4. ホームページに「TOP」、投稿ページに「ブログ」を選んで保存する

便利なプラグイン

- **Show Current Template**：今どのテンプレートファイルが使われているかを管理バーに表示する。慣れて不要になったら削除してよい
- プラグインは入れすぎると不具合や表示速度の低下の原因になる。本当に必要なものだけ入れる（ プラグインの入れすぎを避けるべき理由）

今回の授業のポイント

1. WordPress化すると、管理画面からWebサイトを編集できる
2. 編集したいフォルダは必ず`themes`フォルダに入れ、管理画面で有効化する
3. テンプレート階層で、表示されるファイルの優先順位が決まっている
4. 共通部分はテンプレートパーツファイルに分け、WordPress関数で呼び出す
5. 相対パスは、関数を使った絶対パスにする（エスケープも忘れずに）
6. WebフォントやCSSは`functions.php`にまとめて読み込み、条件分岐で読み込むページを限定できる

参考記事

- WordPressで何ができるか学ぼう
- WordPressを勉強する準備をしよう
- ブログサイトをWordPress化しよう 基盤編
- WordPressのテーマに必要なファイルを押さえよう
- WordPressのテンプレートパーツファイルについて押さえよう
- WordPressを学ぶにあたってPHPの基本を学ぼう
- WordPressでよく使うURL出力関数を学ぼう
- WordPressの特定の関数をエスケープしよう
- WordPressでCSSを読み込もう
- WordPressの条件分岐でページを判別しよう
- テンプレート階層（WordPress公式）
- テンプレート階層と優先順位の早見表
- front-page.phpとhome.phpとindex.phpの違い
- WordPressプラグインの入れすぎを避けるべき理由